

Quick Sort - Randomized

- Avg $O(n \log n)$ $\rightarrow$ Worst $O(n^2)$
- In place sorting
- Divide and Conquer
- Hoare Partition style
- Randomized Quick Sort
 - $\rightarrow$ If helps if the array is already sorted.
 - $\rightarrow$ choose pivot in random
 - $\rightarrow$ can choose recursion depth equally prob.

```
def quick_sort(arr, start, end):
```

```
    if start < end:
```

```
        pivot_idx = random.randint(start, end)
```

```
        nums[start], nums[pivot_idx] = nums[pivot_idx], nums[start]
```

```
        p = hoare_partition(arr, start, end)
```

```
        quick_sort(arr, start, p)
```

```
        quick_sort(arr, p+1, end)
```

```
def hoare_partition(arr, start, end):
```

```
    pivot = arr[start]
```

```
    left = start - 1
```

```
    right = end + 1
```

```
    while True:
```

```
        while True:
```

```
            left += 1
```

```
            if left > end or arr[left] >= pivot:
```

```
                break
```

```
        while True:
```

```
            right -= 1
```

```
            if right < start or arr[right] <= pivot:
```

```
                break
```

```
    if left >= right:
```

```
        return right
```

```
    arr[left], arr[right] = arr[right], arr[left]
```